

A Synopsis
Of
Project
On
**“AI-Powered Automated Pull Request
Verification
& Code Review System”**

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Prof. (Dr.) Akansha Gupta

Department of Computer Science & Engineering

Project TeamID : MP2026CSE83

Submitted By

Name: Sahil Sharma

Enrollment No. : GE-232023583

Name: Ashirwad Kumar

Enrollment No. : GE-232023737

Name: Priyanshu Bisht

Enrollment No. : GE-232023534

CERTIFICATE

This is to certify that the project synopsis entitled “AI-Powered Automated Pull Request Verification & Code Review System” is being submitted by Sahil Sharma (GE-232023583), Ashirwad Kumar (GE-232023737) and Priyanshu Bisht (GE-232023534) for the partial fulfilment of the requirement for the award of the degree of B.Tech – CSE in the Department of Computer Science & Engineering, Graphic Era Deemed to be University, Dehradun, and is a record of his own work carried out under my supervision. The matter embodied in the synopsis has not been submitted in any other university or institution for the award of any degree.

Prof. (Dr.) Akansha Gupta

Prof. Department of Computer Science and Engineering,
Graphic Era Deemed to be University, Dehradun, India.

DECLARATION

We hereby declare that the project synopsis entitled “AI-Powered Automated Pull Request Verification & Code Review System” being submitted for the partial fulfilment of the requirement for the award of the degree of B.Tech (CSE) in the Department of Computer Science & Engineering, Graphic Era Deemed to be University, Dehradun, is my original work and has not been submitted to any other university or institution for the award of any degree or diploma.

Sahil Sharma (GE-232023583)
Ashirwad Kumar (GE-232023737)
Priyanshu Bisht (GE-232023534)

Sahil Sharma
Ashirwad Kumar
Priyanshu Bisht

TABLE OF CONTENTS

S.No. CONTENT	Page No.
Abstract	5
1. Introduction	6
2. Literature Review	7
3. Problem Statement & Objectives	8
4. Proposed Methodology	9
4.1 System Architecture	9
4.2 Four-Level Mental Model	10
4.3 Workflow Stages	11
4.4 Verification Status Model	12
4.5 Component Responsibilities	13
4.6 Reporting and GitHub Integration	14
5. Hardware and Software Requirements	15
6. Expected Outcomes	16
7. Conclusion	17
References	18

ABSTRACT

Code review is an important part of software development, but reviewing every change manually can take time and some issues may still be missed. AI can help developers by finding possible bugs and security problems, but AI-generated suggestions are not always reliable unless they are tested in the actual application.

This project proposes an **AI-Powered Automated Pull Request Verification and Code Review System** that works with GitHub to automatically analyse code changes made through Pull Requests. The system understands the changes in the context of the existing repository and uses specialised AI agents to check for security issues, logical errors, and missing test cases. It can also generate new tests and suggest possible fixes for detected problems.

The proposed fixes and tests are then executed on the complete application inside an isolated Docker environment. If a fix causes failures, the system analyses the results and can attempt to improve the fix within a limited number of attempts. It also compares the original and modified versions of the repository to identify possible regressions.

Finally, the system provides a report containing the detected issues, generated tests, proposed fixes, test results, and verification status. The main goal of the project is to reduce manual effort in code review and make AI-based code suggestions more reliable by validating them through actual execution and testing.

INTRODUCTION

The proposed system is an **AI-Powered Automated Pull Request Verification and Code Review System** integrated with GitHub. Whenever a developer creates or updates a Pull Request, the system receives the code changes and analyses them using information from the existing repository.

The system uses a **Repository Knowledge Base** to understand the structure, dependencies, relationships, and tests of the project. A **Context Retrieval Layer** provides the required information to specialised AI agents. The Security Agent checks for potential security issues, the Logic Agent analyses possible logical errors and generates candidate fixes, and the Test Agent evaluates existing tests and generates additional test cases where required.

The generated tests and candidate fixes are applied to a temporary copy of the complete repository and executed inside an isolated **Docker environment**. The system checks whether the changes work correctly, identifies regressions by comparing the base and Pull Request versions, and can make a limited number of automated repair attempts when tests fail.

After verification, the system generates a detailed report containing the detected issues, test cases, proposed fixes, execution results, and final verification status, which is provided to the developer through the Pull Request.

LITERATURE REVIEW

Recent research has explored the use of Large Language Models (LLMs) for automated software engineering, code review, bug detection, test generation, and program repair. **SWE-bench** introduced a benchmark for evaluating whether AI models can solve real-world GitHub software issues by modifying repository code and running tests. The work showed that solving repository-level software problems requires understanding the wider codebase rather than only the changed file.

1. **SWE-agent** further explored AI agents that can inspect repositories, modify code, and execute programs while solving software engineering tasks. This demonstrated the importance of combining LLM-based reasoning with tools for code navigation and actual code execution.
2. Research such as **SWE-Bench+** highlighted that passing available tests does not always guarantee that a generated patch is correct, showing the importance of stronger test cases and better verification. Other recent research has also investigated the use of LLMs for detecting and repairing security vulnerabilities, demonstrating the potential of AI for security-focused code analysis.
3. Existing tools such as **CodeRabbit** and **Qodo** provide AI-assisted Pull Request review and code understanding. However, these approaches mainly focus on analysing and suggesting changes. The proposed project combines these ideas into a single workflow that performs repository-aware analysis, generates tests and candidate fixes, executes them on the complete application in an isolated environment, and uses the execution results to verify the generated changes.

Thus, the proposed system focuses on moving from **AI-based code suggestions** to **AI-assisted code verification**, where generated findings and fixes are validated through actual testing and execution.

PROBLEM STATEMENT & OBJECTIVES

3.1 Problem Statement

Code review of Pull Requests requires developers to manually examine code changes, identify logical and security issues, and ensure that sufficient tests are available. Although AI-based tools can assist with code review and generate possible fixes, these suggestions may not always be reliable because they are not necessarily verified through actual execution. There is a need for a system that can automatically analyse a Pull Request in the context of the complete repository, generate appropriate tests and fixes, and verify them through actual execution before presenting the results to the developer.

3.2 Objectives

- **Automate Pull Request analysis** by identifying changes and understanding their impact on the existing repository.
- **Detect potential security and logical issues** using specialised AI agents.
- **Analyse existing developer-written tests** and identify important testing gaps.
- **Generate test cases** for new features, identified issues, and important edge cases.
- **Generate candidate code fixes** for detected logical or security problems.
- **Verify generated tests and fixes** by executing them on the complete application inside an isolated Docker environment.
- **Detect regressions** by comparing the behaviour of the original and modified versions of the repository.
- **Perform limited automated repair attempts** when generated fixes fail during verification.
- **Generate a detailed verification report** containing detected issues, tests, proposed fixes, execution results, and final verification status.
- **Reduce manual effort in Pull Request review** while keeping the developer in control of the final decision

PROPOSED METHODOLOGY

The proposed system follows an automated workflow for analysing and verifying GitHub Pull Requests. The methodology consists of the following stages:

Pull Request Detection

A GitHub webhook detects when a Pull Request is created or updated. The system collects the PR details, changed files, code differences, and commit information.

Repository Understanding

The system uses a Repository Knowledge Base containing information about the repository, including files, functions, classes, dependencies, relationships, tests, and code structure. This helps the system understand the PR changes in the context of the complete repository.

1. **Context Retrieval** A Context Retrieval Layer identifies and retrieves the information required for each analysis task. It provides relevant code, dependencies, related functions, and tests to the appropriate AI agent instead of sending the entire repository.
2. **AI-Based Analysis** Specialised AI agents analyse the Pull Request: the Security Agent identifies potential security vulnerabilities, the Logic Agent identifies possible logical errors, behavioural problems, and regressions, and the Test Agent evaluates existing tests and identifies missing test cases.
3. **Test and Fix Generation** Based on the identified issues and hypotheses, the Test Agent generates additional test cases, while the Logic Agent generates candidate fixes for problems that can be automatically addressed.
4. **Isolated Verification** The generated tests and candidate fixes are applied to a temporary copy of the complete repository. The modified application is executed inside an isolated Docker environment along with the existing and newly generated tests.
5. **Failure Analysis and Repair** If the tests fail, the system analyses the execution results and provides the failure information to the relevant AI agent. The system can generate an improved fix and repeat the verification process for a limited number of attempts.
6. **Regression Detection** The system compares the results of the original base version with the Pull Request version to identify whether the new changes have affected previously working functionality.

7. **Report Generation** After the verification process, the system generates a report containing the detected security and logical issues, test cases, proposed fixes, execution results, regressions, repair attempts, and final verification status.
8. **Developer Review** The final results are provided to the developer through the Pull Request. The developer can review the findings and verified fixes before deciding whether to apply or merge the changes.

4.1 System Architecture

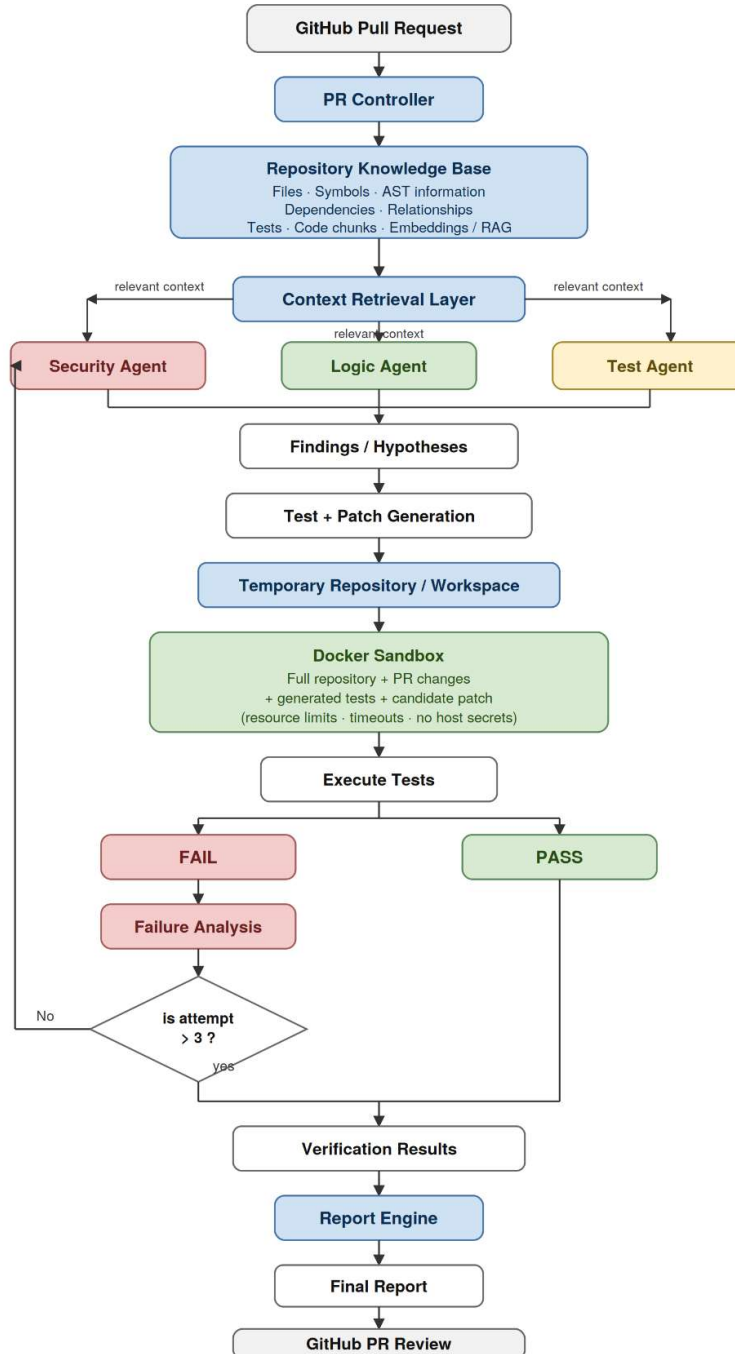


Figure 4.1: Architecture of the AI-Powered PR Verification & Code Review System

4.2 Four-Level Mental Model

The design can be understood as four layered questions:

Level 1 (Repository Knowledge) : “What exists in the repository?”

Level 2 (Context Retrieval) : “What information is relevant right now?”

Level 3 (AI Agents) : “What should we reason about or generate?”

Level 4 (Execution Verification) : “Does the generated result actually work?”

4.3 Workflow Stages

Stage 1 GitHub PR & PR Controller

When a developer creates or updates a PR, GitHub sends a webhook to the backend. The PR Controller receives the repository, PR number, base commit SHA, latest head SHA, PR metadata, changed files, and diff; since GitHub already identifies the directly modified files, retrieval-augmented search is not needed merely to determine which files changed.

Stage 2 Repository Knowledge Base

The repository is indexed at a known-good, human-verified commit. The Knowledge Base stores structured and semantic information sufficient to answer questions such as what a piece of code is, what it depends on, who depends on it, what it affects, and what tests cover it.

KB Component	Information Captured
Repository metadata	Repository, commit SHA, language, dependencies, project structure, configuration
Files	Paths, language, module/file metadata
Symbols	Functions, classes, methods, constants
AST	Functions, classes, imports, decorators, inheritance, calls, assignments, control flow, line numbers
Relationships	Imports, calls, inheritance, usage, modifications, tests
Tests	Test files/functions and implementation mappings
Code chunks	Function, method, class, and module-level chunks
Embeddings / RAG	Optional semantic representations for flexible retrieval

Stage 3 Context Retrieval Layer

Agents do not receive the entire repository; they request the information needed for their current reasoning task, and the Context Retrieval Layer translates those requests into concrete retrieval operations such as `get_file`, `get_symbol`, `get_callers`, `get_callees`, `get_dependents`, `get_tests`, and `get_related_code` over the Knowledge Base. An agent works with sufficient context and can request more when uncertainty remains, subject to a maximum number of investigation iterations so that retrieval cannot continue indefinitely.

Stage 4 Security Agent

The Security Agent analyses the PR diff, changed functions, authentication and authorisation code, database interactions, external inputs, and security-sensitive dependencies. Its findings include location, severity, evidence, and confidence, and are treated as potential issues rather than formal proofs.

Stage 5 Logic Agent

The Logic Agent focuses on behavioural correctness and regressions, examining changed functions, callers, callees, dependencies, modified state, and existing tests, and can generate candidate patches for identified issues.

Stage 6 Test Agent

The Test Agent evaluates whether developer-provided tests actually cover the behaviour introduced by the PR, converts logic and security hypotheses into test hypotheses, and generates candidate test cases, including edge cases and regression tests.

Stage 7 Test & Patch Generation

Candidate tests and candidate fixes produced by the agents are never accepted merely because an LLM generated them; they must pass the subsequent verification process before being reported as trustworthy.

Stage 8 Temporary Workspace and Docker Sandbox

A temporary workspace is created containing the complete repository at the PR's latest commit, the PR changes, existing and AI-generated tests, and a candidate patch. This workspace is executed inside an isolated Docker sandbox with resource limits, execution timeouts, no host secrets, a temporary filesystem, and controlled networking, so that untrusted PR code cannot affect the host system.

Stage 9 Baseline vs PR Verification

The system compares the known-good base commit against the PR commit — for example, 147 of 147 tests passing on the base commit versus 145 of 150 on the PR commit — which distinguishes newly introduced regressions from pre-existing failures.

Stage 10 Failure Analysis & Bounded Repair

If verification fails, the failure information is returned to the Logic Agent, which generates an improved patch for re-verification. A fixed retry limit (for example, three repair attempts) prevents unbounded autonomous modification of the codebase.

4.4 Verification Status Model

Status	Meaning
Detected	Potential issue identified by an agent
Candidate Fix Generated	An LLM-generated patch exists but is not yet trusted
Verification Running	Patch / tests are being executed
Verified	Configured verification checks passed
Verification Failed	Execution exposed a failure
Manual Review Required	Automated attempts could not establish a satisfactory result

4.5 Component Responsibilities

Component	Responsibility
PR Controller	Receives GitHub PR events and coordinates the workflow
Repository Knowledge Base	Stores structured and semantic repository knowledge
Context Retrieval Layer	Retrieves relevant repository evidence for agents
Security Agent	Finds potential security vulnerabilities
Logic Agent	Finds behavioral / logical problems and generates candidate patches
Test Agent	Generates hypotheses / tests and evaluates developer tests
Temporary Workspace	Holds the repository plus candidate changes
Docker Sandbox	Safely executes the application and tests
Verification Controller	Runs tests and manages the retry / failure loop
Report Engine	Produces the final evidence-based report
GitHub Integration	Posts results back to the PR

4.6 Reporting and GitHub Integration

The Report Engine combines PR information, change impact, security findings, logic findings, test analysis, generated tests, candidate patches, execution results, regression information, and retry attempts into a single evidence-based report, concluding with a final recommendation of ready for review, review recommended, or verification failed. This result is surfaced directly on the GitHub PR as a summary and detailed findings, with verified candidate fixes offered for the developer to review or apply; the developer always remains in control of the final merge decision.

HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent.
- **RAM:** Minimum 8 GB, 16 GB recommended.
- **Storage:** Minimum 20 GB of free space.
- **Internet Connection:** Required for GitHub integration and API access.
- **Docker Support:** System capable of running Docker.

5.2 Software Requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Programming Language:** Python 3.11+
- **Backend Framework:** FastAPI
- **AI Orchestration:** LangGraph
- **LLM API:** OpenAI API or another compatible LLM provider
- **Repository Integration:** GitHub API and GitHub Webhooks
- **Code Analysis:** Python AST
- **Testing Framework:** pytest
- **Containerization:** Docker
- **Database:** SQLite with SQLAlchemy
- **Semantic Retrieval:** Embeddings with a suitable vector storage/retrieval system
- **Dashboard:** Streamlit
- **Version Control:** Git and GitHub
- **Development Environment:** VS Code or any suitable Python IDE

EXPECTED OUTCOMES

The proposed system is expected to provide an automated and repository-aware Pull Request review and verification process. The expected outcomes are:

- **Automatic Pull Request Analysis:** The system will analyse code changes and understand their impact on the existing repository.
- **Security and Logic Issue Detection:** Potential security vulnerabilities and logical or behavioural issues will be identified using specialised AI agents.
- **Automated Test Generation:** The system will evaluate existing tests and generate additional test cases for important scenarios and identified issues.
- **Candidate Fix Generation:** The system will generate possible fixes for detected problems and validate them through execution.
- **Automated Verification:** Generated tests and fixes will be executed on the complete application in an isolated Docker environment.
- **Regression Detection:** The system will identify whether new changes affect previously working functionality by comparing the base and Pull Request versions.
- **Iterative Repair:** Failed fixes will be analysed and improved through a limited number of automated attempts.
- **Evidence-Based Reporting:** A detailed report will provide detected issues, generated tests, proposed fixes, execution results, regressions, and verification status.
- **Reduced Manual Effort:** The system will reduce the amount of repetitive work required from developers during Pull Request review while keeping the final decision under developer control.
- **More Reliable AI Assistance:** AI-generated findings and fixes will be supported by actual test and execution results rather than being treated as verified solely on the basis of LLM output.

CONCLUSION

The proposed **AI-Powered Automated Pull Request Verification and Code Review System** aims to make the code review process faster, more reliable, and less dependent on manual effort. The system combines repository understanding, context-based AI analysis, automated test generation, candidate fix generation, and actual code execution in an isolated Docker environment.

By using specialised Security, Logic, and Test agents, the system can identify potential issues, generate appropriate tests and fixes, and verify them through real execution. The comparison between the original and modified repository also helps identify possible regressions. Finally, the system provides developers with a detailed report containing the findings and verification results.

Overall, the project aims to move AI-assisted code review from simply **suggesting possible problems and fixes** to **testing and verifying those suggestions with actual execution evidence**, while keeping the developer responsible for the final decision.

REFERENCES

- [1] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in International Conference on Learning Representations (ICLR), 2024.
- [2] A. Yang et al., “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” in Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [3] A. Liu et al., “SWE-Bench+: Enhanced Coding Benchmark for LLMs,” arXiv preprint arXiv:2410.06992, 2024.
- [4] OpenAI, “Introducing SWE-bench Verified,” 2024. Available: <https://openai.com/index/introducing-swe-bench-verified/>
- [5] “SWE-bench,” GitHub Repository. Available: <https://github.com/SWE-bench/SWE-bench>
- [6] “SWE-agent,” GitHub Repository. Available: <https://github.com/SWE-agent/SWE-agent>
- [7] “Secure coding with AI – from detection to repair,” Empirical Software Engineering, 2026.
- [8] CodeRabbit, “AI Code Reviews,” Available: <https://www.coderabbit.ai/>
- [9] Qodo, “AI Code Review and Quality Platform,” Available: <https://www.qodo.ai/>
- [10] Docker, “Docker Documentation,” Available: <https://docs.docker.com/>
- [11] GitHub, “GitHub REST API Documentation,” Available: <https://docs.github.com/en/rest>
- [12] LangChain, “LangGraph Documentation,” Available: <https://langchain-ai.github.io/langgraph/>
- [13] pytest, “pytest Documentation,” Available: <https://docs.pytest.org/>
- [14] Python Software Foundation, “ast — Abstract Syntax Trees,” Python Documentation. Available: <https://docs.python.org/3/library/ast.html>